



Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

Arabic Abstractive Text Summarisation using Seq2Seq with Attention

By

Mohammad ALADDASI 20220630, Hamza Rashdan 20230580, Saif AlSharkasi
20230594

Supervised by

Dr Mohammad Azzeh

Natural Language Processing (14351)

PRINCESS SUMAYA UNIVERSITY FOR TECHNOLOGY

Amman, Jordan

Spring 2025/2026

Table of Contents

Github Repository.....	3
Introduction.....	3
Dataset & Preprocessing.....	3
Techniques & Model Architecture.....	5
Results.....	6
Evaluating Methods.....	7
Limitations.....	9
Future Work.....	10
References (APA).....	11

Github Repository

<https://github.com/Mohammad-ALADDASI/arabic-text-summarisation-seq2seq>

Introduction

Text summarisation is an important task in the field of Natural Language Processing (NLP) that focuses on generating concise and meaningful summaries from long textual documents. Since Arabic language is a morphologically rich and complex language, building effective Arabic text summarisation systems remains a difficult task. Challenges such as limited Arabic NLP resources, diverse writing styles, diacritics, and linguistic ambiguity which affect the quality and coherence of generated summaries. These issues may reduce the semantic accuracy and lexical consistency of the summarisation model if they are not properly addressed.

There are several common methods for handling text summarisation tasks, such as statistical methods, traditional machine learning algorithms, and deep learning architectures. Recent advancements in NLP, especially through deep learning, Sequence-to-Sequence (Seq2Seq) models combined with attention mechanisms, have shown promising results in abstractive summarisation tasks.

In this project, an Arabic abstractive text summarisation system based on a Seq2Seq architecture with Bahdanau Attention will be implemented, trained, and evaluated using multiple Arabic summarisation datasets. The performance of the model will then be analysed using ROUGE evaluation metrics and qualitative comparisons between generated and reference summaries.

Dataset & Preprocessing

This project uses multiple Arabic text summarisation datasets to train and evaluate an Arabic abstractive summarisation model. The goal of the dataset stage is to prepare article–summary pairs, where the article is used as the input text and the summary is used as the target output that the model learns to generate. The datasets used are SumArabic, AraSum, Arabic Text Summarisation Dataset, and Egyptian Arabic Text Summarisation Dataset. The raw data was not stored in one single format. Some files were stored as .csv, some as .jsonl, and one Egyptian dataset file was stored as .parquet. Because of this, the preprocessing stage had to standardise the different datasets into the same article–summary format before they could be merged and used for model training.

The first dataset is SumArabic, which was divided into four JSONL files: training, validation, testing, and out-of-distribution data. The raw SumArabic files contain many columns, but we focus on the ‘text’ column which represents the article, and the ‘headline’ column for the summary. After preprocessing, SumArabic was saved as four processed CSV files: SumArabic-train.csv, SumArabic-valid.csv, SumArabic-test.csv, and SumArabic-ood.csv.

The second dataset is AraSum. The main AraSum file used in the processed dataset is AbsArSumCorpus_DW_article tab lead.csv. This file is tab-separated and contains article

text with its corresponding lead or summary. After cleaning, it was saved as cleaned_AraSum_AbsArSumCorpus_DW_article tab lead.csv.

The third dataset is the Arabic Text Summarisation Dataset. This dataset contains files such as data.csv and summarisdataset.csv. The processed file used in the final training data is cleaned_Arabic_Text_Summarisation_data.csv. In this dataset, the original article text is stored in the Processed Text column, while the reference summary is stored in the summariser column.

The fourth dataset is the Egyptian Arabic Text Summarisation Dataset. The raw file is stored as a Parquet file named train-00000-of-00001.parquet. After preprocessing, it was converted into a CSV file named cleaned_Egyptian_Text_Summarisation_data_train-00000-of-00001.csv. In this processed file, the original text column is text, and the summary column is summarised_text.

All datasets were converted into a common article–summary pair format. This is necessary because the Seq2Seq model requires one input sequence and one target sequence for each training example. The article is the long input sequence, and the summary is the shorter output sequence.

The processed files contain standard cleaned columns: clean_article, clean_summary, article_tokens, summary_tokens, article_len and summary_len. Some processed files also kept their original columns. However, the model training uses clean_article and clean_summary, which are the required columns for training.

After preprocessing, the final processed folder contains seven cleaned CSV files. These files are:

- cleaned_AraSum_AbsArSumCorpus_DW_article tab lead.csv, 49579 rows
- cleaned_Arabic_Text_Summarisation_data.csv, 5010 rows
- cleaned_Egyptian_Text_Summarisation_data_train-00000-of-00001.csv, 3689 rows
- SumArabic-train.csv, 75817 rows
- SumArabic-test.csv, 4174 rows
- SumArabic-valid.csv, 4121 rows
- SumArabic-ood.csv, 652 rows

After removing empty or missing article–summary pairs and removing duplicate article–summary pairs, we have a final combined dataset of 142,802 samples out of the original 143,042 total samples. The final dataset was then split into training, validation, and testing sets using an (80,10,10) split (114,241 training samples, 14,280 validation samples, and 14,281 test samples).

Arabic text requires special preprocessing because the same word can appear in different written forms. The project preprocessing pipeline includes removing URLs and special characters, Arabic normalisation, removing diacritics, tokenisation, and removing duplicated or empty samples.

Normalisation was used to reduce unnecessary spelling variation in Arabic text (اقتصاد to إقتصاد). This helps the model treat words with the same meaning as the same token instead of learning them as separate words.

Arabic diacritics (تشكيل) were removed because they are not always present in real Arabic text and can increase vocabulary size unnecessarily. Makes the text more consistent and reduces sparsity during tokenisation.

After cleaning, the text was tokenised by splitting the article and summary into word tokens. These tokens were then used to calculate article and summary lengths. Later, during

model training, the text was converted into numerical sequences so that it could be passed into the Seq2Seq neural network.

The processed dataset contains both long and short articles. The overall average article length is about 150.92 tokens, while the average summary length is about 17.81 tokens.

Table 1: A statistical summary table for the average length of every dataset used

Dataset file	Avg. article length	Avg. summary length
AraSum	367.24	33.27
Arabic Text Summarisation	84.09	30.38
Egyptian Text Summarisation	48.04	17.95
SumArabic-train	32.82	8.02
SumArabic-valid	32.86	8.01
SumArabic-test	32.87	7.98
SumArabic-ood	31.29	8.30

Before training, we also applied length filtering to avoid extremely long samples because they make training slower and high memory usage. The model kept articles between 5 and 300 tokens and summaries between 2 and 40 tokens. After this filtering, the notebook reports 92,751 training samples, 11,642 validation samples, and 11,580 test samples.

Overall, the dataset preparation stage transformed different Arabic summarisation datasets into one unified cleaned dataset. The most important steps were standardising column names, converting all data into article–summary pairs, cleaning Arabic text, removing empty and duplicate samples, calculating token lengths, and splitting the data for training, validation, and testing.

Techniques & Model Architecture

To tackle the task of Arabic abstractive text summarisation, this project employs a Sequence-to-Sequence (Seq2Seq) neural network architecture. The Seq2Seq framework is

uniquely suited for conditional text generation tasks like summarisation, where an input sequence of variable length (the source article) must be mapped to an output sequence of variable length (the summary). The model is composed of an Encoder, an Attention mechanism, and a Decoder, with a total trainable parameter count of approximately 37.7 million. The Encoder is responsible for reading the input Arabic article and compressing its semantic meaning into a continuous vector representation. It uses a Bidirectional Gated Recurrent Unit (BiGRU) consisting of 2 stacked layers. By processing the input sequence in both forward and backward directions, the BiGRU captures contextual information from both the past and future tokens at every time step. The input tokens are mapped to a 128-dimensional embedding space using a vocabulary restricted to the top 30,000 most frequent words, and the hidden state dimension of the GRU is set to 256. To mitigate overfitting, a dropout rate of 0.4 is applied between the GRU layers. The final hidden states from both the forward and backward passes are concatenated and passed through a fully connected linear layer to initialise the Decoder.

Because a traditional Seq2Seq model suffers from an information bottleneck when compressing the input into a single fixed-size context vector, we implemented Bahdanau (additive) Attention. This mechanism calculates an alignment score between the current hidden state of the Decoder and all hidden states of the Encoder, converting them into attention weights via a softmax function. These weights are used to compute a dynamic context vector, allowing the Decoder to focus on the most relevant parts of the source article at each step of the summary generation. The Decoder generates the final Arabic summary one token at a time using a unidirectional GRU, which is also configured with 2 stacked layers to match the depth of the Encoder. It uses an embedding dimension of 128 and a hidden dimension of 256. At each time step, the input to the Decoder's GRU is a concatenation of the previously generated token's embedding and the current attention context vector, before a final linear layer projects the output back into the 30,000-dimensional vocabulary space.

The training pipeline was designed to ensure stable convergence and prevent overfitting. The model was trained using the Adam optimiser with a learning rate of 0.0005 and a Cross-Entropy Loss function configured to ignore padding tokens. Gradient clipping was applied with a maximum threshold of 1.0, and maximum sequence lengths were capped at 300 tokens for articles and 42 tokens for summaries. During training, a Teacher Forcing ratio of 0.5 was used, meaning that 50% of the time, the Decoder was fed the actual ground-truth target token as the next input to speed up early convergence while forcing the model to learn to recover from mistakes. Finally, an Early Stopping mechanism was implemented with a patience of 3 epochs; training was halted early at Epoch 7, with the best model checkpoint saved at Epoch 4 achieving a validation loss of 6.47.

Results

The Seq2Seq with Attention model was trained on a collection of Arabic news summarisation datasets after applying preprocessing and dataset cleaning. During training, both the training loss and validation loss were monitored to evaluate the learning behaviour of the model and detect overfitting.

The model showed consistent learning behaviour during the first few epochs. The training loss decreased slowly from approximately 7.49 in the first epoch to 6.13 by the third

epoch. At the same time, the validation loss also improved from 6.65 to 6.31, indicating that the model was learning successfully meaningful representations from the training data.

The best model performance was achieved at epoch 3, where the validation loss reached approximately 6.315. After this point, the validation loss began to increase while the training loss continued to decrease. This behaviour indicates that the model started to overfit the training data. To prevent further degradation in generalisation performance, an early stopping mechanism was used, which terminated training after three consecutive epochs without improvement.

The following figure illustrates the relationship between training loss and validation loss across epochs.

- Training loss continuously decreased during training.
- Validation loss decreased initially, then gradually increased after epoch 3.
- The divergence between the two curves demonstrates the beginning of overfitting.

The model was able to capture important topic-related words and entities from Arabic news articles. In many cases, the generated summaries contained keywords strongly related to the article topic, such as country names, organisations, and important events. However, the generated summaries still suffered from several limitations, including repetition, incomplete sentence structure, and occasional topic drift.

Examples of generated summaries showed that the model often produced semantically related phrases but struggled to generate fully coherent and factually accurate abstractive summaries. For example, articles discussing health or COVID-19 were frequently summarised using related keywords such as “الصحة”, “كورونا”, and city names, even when the exact wording differed from the reference summary.

All in all, the results demonstrate that the Seq2Seq with Attention architecture can learn basic semantic relationships in Arabic summarisation tasks, but its performance remains limited compared to modern Transformer-based architectures. Nevertheless, the model provides a functional baseline for Arabic abstractive text summarisation and highlights the challenges associated with Arabic NLP tasks, particularly in low-resource and morphologically rich settings.

Evaluating Methods

The model was evaluated using ROUGE metrics. ROUGE compares the generated summary with a human-written reference summary and measures the amount of overlap between them. In this project, we used ROUGE-1, ROUGE-2, and ROUGE-L. Higher ROUGE scores indicate better summarisation quality.

ROUGE-1 measures unigram overlaps between the generated summary and the reference summary. It checks how many individual words are in both the generated output and the correct summary. A higher ROUGE-1 score means the model is capturing more important topic words.

ROUGE-2 measures bigram overlap. It checks whether two-word sequences in the generated summary match two-word sequences in the reference summary. ROUGE-2 is stricter than ROUGE-1 because the model must generate words in the correct order, not only individual correct words.

ROUGE-L measures the longest common subsequence between the generated and reference summaries. This means it gives credit when the generated summary follows a similar word order or structure to the reference summary, even if not all words match exactly.

For each ROUGE metric, the project reports precision, recall, and F1-score.

- Precision shows whether the generated summary contains useful words instead of unrelated words.
- Recall shows whether the model captured the essential information from the reference summary.
- F1-score combines both precision and recall into one score.

The model was evaluated on 200 test samples using Arabic-safe ROUGE evaluation. The outputs were the following scores:

Table 2: Evaluation Metrics Summary Table for the Trained Model

Metric	Precision	Recall	F1-score
ROUGE-1	0.3186	0.2066	0.2393
ROUGE-2	0.0705	0.0517	0.0565
ROUGE-L	0.2659	0.1803	0.2053

The evaluation shows that the model learned useful summarisation behavior. The ROUGE-1 F1-score of 0.2393 tells us that the model can identify some important words from the article. This is useful because summarisation depends on recognising the main topic of the input text. The ROUGE-2 F1-score is 0.0565, this means the model has difficulty generating correct multi-word expressions. The ROUGE-L F1-score is 0.2053, which means that the model sometimes follows part of the reference summary sequence, but it does not consistently generate summaries with the same meaning and structure as the human-written summaries. Therefore, the model is better at topic capture than at accurate sentence generation.

One strength of the model is that it captures the general topic of the article. For example, in a sample, the article talks about Pfiser and BioNTech submitting an emergency authorisation request for their COVID-19 vaccine. The generated summary was “شركة بيونتيك” “تؤكد رسميا ضد لقاح كورونا”. This output is not perfect, but it includes important topic words such

as BioNTech, vaccine, and Corona. This shows that the model can sometimes identify the main subject of the article.

The main weakness of the model is phrase generation. Many generated summaries contain some relevant words but do not form a complete or accurate sentence, hence the low ROUGE-2 score. Another weakness is that the model sometimes generates unrelated or incorrect words and does not capture the meaning of the article. The model also sometimes captures the general domain but misses the specific event.

Here are some example outputs:

Article topic	Reference summary	Generated summary	Interpretation
Dubai customs and marijuana oil seizures	ضبطيه لاحتياط دخول مخدر زيت المارغوانا الى الدوله	ارتفاع في دبي خلال العام الماضي	Captures the general topic, but incomplete
French digital tax	فرض ضريبه على مجموعات رقميه عملاقه	البرلمان الاوروبي على البريطاني للبرلمان	Captures political topic but misses main event
Nakba Day during COVID-19	الفلسطينيون يحيون يوم النكبه بدون فعاليات لاول مره بسبب كورونا	تايلاند اليوم من في	Incorrect and unrelated

These examples show the difference between correct and incorrect outputs. Correct or partially correct outputs usually contain important topic words from the article (topic capture), such as “كورونا”, “لقاح”, “بيونتيك”, or “دبي”. Incorrect outputs either use unrelated words or produce incomplete Arabic phrases.

Overall, the evaluation shows that the Seq2Seq attention model can sometimes identify the topic of an Arabic news article, but it struggles to generate fluent and accurate summaries. The strongest result is ROUGE-1, which means the model is better at predicting individual important words. The weakest result is ROUGE-2, which means the model has difficulty generating correct phrases and word sequences.

Limitations

While the Seq2Seq model with Bahdanau attention successfully learns to construct coherent Arabic sentences, the generated outputs exhibit distinct failure modes. Evaluating the

qualitative samples and the resulting ROUGE scores (ROUGE-1 F1 of 0.2393 and ROUGE-L F1 of 0.2053) reveals two primary limitations: text repetition and hallucination. A common issue observed during the generation phase is the model getting stuck in repetitive loops, such as repeating the same word or bigram multiple times (e.g., repeating "ابوظبي"). This is a well-documented limitation of standard Recurrent Neural Networks (RNNs) combined with greedy decoding strategies. Because the model relies heavily on its immediate local context, predicting a word once increases the probability of predicting it again in the next step. To mitigate this, a custom post-processing function with repetition penalties and bigram blocking was introduced to forcefully stop immediate loops, but the core architectural limitation remains that GRUs struggle to maintain long-term coherence over extended generation steps without advanced algorithms like Beam Search.

Additionally, the model frequently hallucinates entities or drifts entirely from the original topic of the text. For example, an article discussing Palestinians and the Coronavirus was summarised by the model with a completely unrelated mention of Thailand, and an article about the French Parliament was summarised with references to the European and British Parliaments. These hallucinations stem from both architectural and data-related constraints. Standard Seq2Seq models lack a copy mechanism, meaning that when faced with out-of-vocabulary words or rare named entities, the language model guesses a highly probable but contextually incorrect entity from its vocabulary rather than copying from the source. Furthermore, abstractive summarisation datasets often contain high variance and noise; if reference summaries in the training data contain external information or opinions not strictly present in the source articles, the model learns to guess external facts rather than relying purely on the input text, leading to severe topic drift.

Future Work

Several improvements can be made to enhance the performance and robustness of the Arabic text summarisation system. More advanced transformer-based architectures such as BART, T5, and AraBART can be incorporated to improve semantic understanding, contextual learning, and summary coherence. These models have shown superior performance in abstractive summarisation tasks compared to traditional recurrent neural network architectures. In addition, more specialised Arabic NLP tools can be integrated into the preprocessing pipeline, such as CAMeL Tools and Farasa, to improve tokenisation, stemming, morphological analysis, and handling of Arabic linguistic variations. This can help reduce ambiguity and improve the lexical and semantic quality of generated summaries.

Finally, more comprehensive evaluation techniques can be explored in addition to ROUGE metrics. Semantic similarity measures such as BERTScore and human evaluation methods can provide deeper insight into the factual correctness, coherence, and readability of generated summaries. Future improvements may also include reinforcement learning techniques and coverage mechanisms to reduce repetition and improve semantic alignment between generated and reference summaries.

References (IEEE)

- [1] ALADDASI, M. O., Rashdan, H., & Sharkasi, S. (2026). Arabic abstractive text summarisation using Seq2Seq with attention [Computer software]. GitHub.
<https://github.com/your-repository-link>
- [2] DeepLearning.AI. (2026). *Natural Language Processing Specialisation*. DeepLearning.AI. Retrieved May 6, 2026, from [Natural Language Processing Specialisation](#)
- [3] Azzeh, M. (2026). *Natural Language Processing course slides*. Princess Sumaya University for Technology (PSUT).
- [4] Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition* (3rd ed.). Prentice Hall.